Awaaz Admin Web

Project Handover Document

Version 1.0 | June 17, 2026 | Confidential

Quick Reference — Application Routes

Route	Screen
/	Splash
/login	Login
/register	Register
/forgotPassword	Forgot Password
/otpScreen	OTP Verification
/resetPassword	Reset Password
/onBoarding	Onboarding
/dashboard	Main Dashboard (Events default tab)
/rescue	Rescue Dashboard
/event/attachEvent	Attach Event Media
/event/eventDetails	Event Details
/rescue/rescueDetails	Rescue Details
/general/generalDetails	General Post Details
/superAdminDashboard	Super Admin
/event/userProfile/:userId	User Profile

1. Project Overview

Project Name

Awaaz Operator — Admin Web Panel (package: eagle_eye_admin)

Purpose and Goals

The Awaaz Operator Admin Web Panel is a Flutter-based web application that enables authorized operators and administrators to manage content and users for the Awaaz mobile platform. Operators create and moderate Events, Rescue posts, and General posts; review user reports; manage app users; and (for Super Admins) approve or reject admin registrations. The panel is optimized for desktop browsers with responsive support for tablet and mobile widths.

Target Platforms

- Primary: Web (Flutter Web) — deployed via Firebase Hosting
- Secondary (codebase supports): Android, iOS, macOS, Windows
- Admin handover scope: Web deployment and browser-based operator workflows

Key Features Summary

- Email/password and Google Sign-In authentication
- Multi-module dashboard: Events, Rescue, Reports, User Management, Super Admin, General Posts
- Event/Rescue/General post creation, drafts, timelines, and media attachments
- Location-based filtering with radius and calendar filters
- Category and subcategory management
- Reported posts, comments, and users moderation
- App user search, profile view, block/unblock
- Super Admin approval workflow for admin users
- Push notifications (Firebase Cloud Messaging) on supported platforms
- Dark-themed operator UI (1920×1080 design baseline)

2. Technology Stack

Flutter Version

SDK constraint: ^3.5.4 (pubspec.yaml). Verified environment: Flutter 3.44.1 (stable)

Dart Version

SDK constraint: ^3.5.4. Verified environment: Dart 3.12.1

State Management

GetX (get: 4.6.6) — primary pattern for Admin Web:

- GetxController classes in lib/controller/
- GetBuilder for UI rebuilds
- Rx types (e.g., RxBool) for reactive state in select controllers
- Get.put / Get.find for dependency injection

Note: Legacy consumer-app code under lib/presentation/ uses Bloc/Cubit but is NOT part of the active Admin Web flow.

Key Packages and Purpose

- go_router (^7.0.0) — Declarative routing with GetMaterialApp.router
- dio (^5.4.3+1) — HTTP client (secondary; primary API via logic_go_network)
- logic_go_network (Git) — RestClient wrapper, APIType (public/protected), Failure handling
- get_storage (^2.1.1) — Local persistence (JWT token, user prefs, tab state)
- flutter_screenutil (^5.9.3) — Responsive scaling (design size 1920×1080)
- firebase_core, firebase_auth, firebase_messaging — Firebase integration
- google_sign_in — Google OAuth for admin login
- flutter_svg, lottie — UI assets and animations
- cached_network_image — Network image caching
- file_picker, image_picker, image_picker_web — Media upload (web-aware)
- geolocator, geocoding — Location services
- table_calendar — Date filtering
- dropdown_button2 — Enhanced dropdowns
- toastification — Toast notifications
- connectivity_plus, internet_connection_checker — Network connectivity
- intl — Date/number formatting
- video_player, get_thumbnail_video — Video preview and thumbnails
- flutter_gen_runner — Auto-generated asset references (lib/gen/)

3. Project Architecture

Architecture Pattern

Feature-oriented MVC with Repository pattern (not full Clean Architecture).

Layers:

- Presentation: lib/page/ (screens + components), lib/widget/ (shared UI)
- Controller: lib/controller/ (GetX controllers — business logic & UI state)
- Data: lib/api/repository/ (API calls), lib/model/ (DTOs)
- Services: lib/service/ (storage, location)
- Routing: lib/route/ (GoRouter config, navigation helpers)
- Theme/Utils: lib/theme/, lib/utils/

Folder Structure Strategy

Admin Web code is organized by feature under lib/page/ with matching controllers and repositories. Shared widgets live in lib/widget/. API endpoints are centralized in lib/api/api_endpoint.dart. Cross-cutting concerns (auth headers, base URL) are in lib/utils/app_constants.dart and lib/api/api_endpoint.dart.

Code Organization Notes

The repository contains legacy consumer-app modules under lib/presentation/, lib/core/, and lib/data/ (Bloc/Cubit architecture). These directories are remnants from a shared monorepo and are NOT wired into main.dart or AppRoutes for the Admin Web application. Future maintainers should treat lib/page/, lib/controller/, lib/api/, and lib/model/ as the authoritative Admin Web codebase.

4. Folder Structure Breakdown

lib/

Root application source. Entry point: main.dart

lib/page/

Feature screens and UI components:

- dashboard_screen/ — Main shell with side navigation
- event_screen/ — Event CRUD, drafts, details, attachments
- rescue_screen/ — Rescue posts and updates
- general_screen/ — General post management
- reports_screen/ — Moderation of reported content
- user_management_screen/ — App user listing and actions
- super_admin_dashboard/ — Admin approval workflow
- login_screen/, register_screen/, forgot_password_screen.dart/ — Auth flows
- splash_screen/, onboarding_screen/ — App entry and first-time setup

lib/controller/

17 GetX controllers — one per major feature (event, rescue, login, dashboard, etc.)

lib/api/

- api_endpoint.dart — All REST endpoint path constants and auth headers
- repository/ — Auth, Event, Rescue, General, Report, User, SuperAdmin repositories
- interceptor/token_interceptor.dart — Dio Bearer token interceptor (currently not attached in ApiControllerMain)
- connectivity_check/ — Network connectivity monitoring

lib/model/

22 JSON-serializable data models (events, users, reports, categories, etc.)

lib/service/

- storage/ — GetStorage wrapper (StorageService, StorageKeys)
- location/ — Location service helpers

lib/widget/

Shared UI: drawer, buttons, dialogs, text fields, toast, network image loader

lib/theme/

AppColors, ProgressLoader, theme constants

lib/route/

app_route.dart (GoRouter), navigator_route.dart (navigation helpers)

lib/utils/

app_constants.dart (base URLs), responsive_utils.dart, app_functions.dart, location_utils.dart

lib/gen/

Auto-generated assets.gen.dart and fonts.gen.dart (flutter_gen)

lib/core/

Not Available for Admin Web — legacy consumer app shared widgets/utilities

lib/presentation/

Not Available for Admin Web — legacy consumer app Bloc/Cubit screens

lib/data/

Not Available for Admin Web — legacy consumer app models

lib/domain/

Not Available — Clean Architecture domain layer not implemented

5. Setup & Installation Guide

Prerequisites

- Flutter SDK 3.5.4 or higher (3.44.1 recommended)
- Dart SDK 3.5.4+
- IDE: Android Studio, VS Code, or Cursor with Flutter/Dart extensions
- Git (for logic_go_network dependency from Bitbucket)
- Chrome or Edge browser for web development
- Firebase CLI (for web deployment): npm install -g firebase-tools
- Node.js (for Firebase CLI)

Install Dependencies

1. Clone the repository
2. cd to project root (Awaaz/)
3. Run: flutter pub get
4. Run code generation (if assets changed): dart run build_runner build --delete-conflicting-outputs
5. Ensure Bitbucket access for logic_go_network Git dependency

Run the Project (Web)

Development:

flutter run -d chrome

Release build (local):

flutter build web --web-renderer html --release

Serve locally:

cd build/web && python -m http.server 8080

Environment Configuration

- Base API URL: lib/utils/app_constants.dart

- Admin API: https://awaazeye.com/admin/v1

- User API: https://awaazeye.com/api/v1

- Firebase: lib/firebase_options.dart (auto-generated via FlutterFire CLI)
- Web Firebase config: web/index.html (Firebase JS SDK)
- Google Sign-In: web/index.html meta tag (google-signin-client_id)
- config/secrets.json — present in repo (verify contents are not committed to public repos)
- android/secrets.properties, ios/Flutter/Secrets.xcconfig — platform secrets
- Radar.io API key: hardcoded in dashboard_controller.dart (location autocomplete) — should be moved to env config
- No .env file pattern currently implemented; URLs and keys are in source files

6. Build & Deployment

Web Build

Command (from README):

flutter build web --web-renderer html --release

Output directory: build/web/

Renderer: HTML (set in web/index.html: window.flutterWebRenderer = "html")

Firebase Hosting Deployment

1. flutter build web --web-renderer html --release

2. firebase login

3. firebase deploy --only hosting

Configuration:

- firebase.json — public: build/web, SPA rewrites to index.html
- .firebaserc — project: eagle-eye-oprator

APK / IPA Builds

Android APK:

flutter build apk --release

Android App Bundle:

flutter build appbundle --release

iOS IPA:

flutter build ipa --release

(Requires macOS, Xcode, Apple Developer account)

Note: Admin Web primary delivery is browser-based; mobile builds exist but are secondary.

Release Configuration

- App version: pubspec.yaml — version: 1.0.0+1
- Android: android/app/build.gradle (applicationId, signing)
- iOS: ios/Runner/Info.plist, Xcode project settings
- Web title: "Awaaz Operator" (main.dart, web/index.html)
- Firebase project ID: eagle-eye-oprator

7. State Management Explanation

How State Is Handled

Admin Web uses GetX controllers as the single source of truth per feature. Controllers extend GetxController and call update() to trigger GetBuilder rebuilds. Some reactive fields use RxBool/Rx types with .obs. Controllers are registered via Get.put() at screen level or dashboard initialization.

Controller Lifecycle

1. Screen initState or GetBuilder initState calls Get.put(Controller)
2. Controller fetches data via Repository
3. Controller updates local lists/fields and calls update()
4. GetBuilder rebuilds bound UI widgets
5. User actions call controller methods → API → update() → UI refresh

Example Flow: Login

UI (LoginScreen)

→ User taps Login

→ LoginController.loginWithEmailNPasssword(context, loader)

→ AuthRepository.loginWithEmail() via restClient.post(APIType.public)

→ ResponseModel parsed → StorageService.storeToken()

→ NavigatorRoute.navigateToRemoveUntil(AppRoutes.event, context)

→ DashboardScreen loads with EventController, DashboardController, etc.

Example Flow: Event List

UI (EventDashboardView)

→ GetBuilder initState: eventController.getEventsList()

→ EventRepository.filterEvents() → REST API

→ eventController.filterEvents populated → update()

→ EventPostCardView renders list with pagination (pageNumber)

Bloc/Cubit

Not used in Admin Web active codebase. Present only in legacy lib/presentation/ modules.

8. API Integration

API Structure

- Primary client: logic_go_network RestClient (initialized in ApiControllerMain.init())
- Base URL: https://awaazeye.com/admin/v1
- Endpoint paths: lib/api/api_endpoint.dart
- Repositories: lib/api/repository/*.dart

- Response wrapper: ResponseModel (status, message, body)
- Secondary Dio instance in ApiControllerMain (baseUrl: https://awaazeye.com) — used for ad-hoc calls

Base URL Handling

lib/utils/app_constants.dart:

String baseUrl = 'https://awaazeye.com/admin/v1';

String userBaseUrl = 'https://awaazeye.com/api/v1';

Repositories concatenate: "$baseUrl/${ApiEndpoint.}"

Authentication Flow

1. POST auth/login/email — returns JWT token and user object
2. Token stored via StorageService.storeToken()
3. Protected requests: requestHeader(APIType.protected) adds Bearer token
4. Google Sign-In: Firebase Auth → POST auth/login-and-register/google
5. Token expiry: AppFunctions.onTokenExpire() clears storage, preserves saved credentials, redirects to login
6. Splash screen checks StorageService.getToken() → login or dashboard

Error Handling Strategy

- Repository methods catch Failure from logic_go_network
- Error strings parsed from exception message
- Returns ResponseModel(status: false, message: error)
- UI shows toast via showToast() with AppColors.red for errors
- 401 handling partially implemented in AuthRepository (onTokenExpire commented in login)
- ProgressLoader shown/hidden around async operations
- NetworkConnectionService monitors connectivity (limited UI feedback)

Key API Modules

Auth: login, register, forgot-password, OTP, reset-password, Google sign-in

Events: CRUD, filter, drafts, timeline, file upload, status updates

Rescue: rescue updates, pending count, status updates

General: general post CRUD and drafts

Reports: reported posts, comments, users; delete comment; block user

Users: app user list, profile, FCM token update, radius update

Super Admin: admin list, approve/reject admin status

Categories/Reactions: list, create category/subcategory

9. Key Modules / Features

Authentication Module

{'desc': 'Email/password login, registration, forgot password (OTP), reset password, Google Sign-In.', 'files': 'lib/page/login_screen/, register_screen/, forgot_password_screen.dart/, lib/controller/login_controller.dart, register_controller.dart, forgot_password_controller.dart, lib/api/repository/auth_repository.dart', 'flow': 'Splash → Login/Register → Token stored → Dashboard. Forgot password: email → OTP → reset.'}

Dashboard Shell

{'desc': 'Main container with side drawer navigation and tab switching between modules.', 'files': 'lib/page/dashboard_screen/dashboard_screen.dart, lib/widget/drawer.dart, lib/controller/dashboard_controller.dart', 'flow': 'DashboardController.selectedTab (0=Event, 1=Rescue,

2=Reports, 3=User Mgmt, 4=Super Admin, 5=General). Desktop shows permanent drawer; mobile uses hamburger menu.'}

Events Module

{'desc': 'Create, edit, filter, draft, publish events with categories, location, timeline, and media attachments.', 'files': 'lib/page/event_screen/*, lib/controller/event_controller.dart, event_detail_controller.dart, lib/api/repository/event_repository.dart', 'flow': 'Event list (filter by location/radius/calendar) → Create/Edit → Preview → Publish. Sub-tabs: Active posts (1.1), Drafts. Routes: /dashboard, /event/attachEvent, /event/eventDetails.'}

Rescue Module

{'desc': 'Manage rescue posts, updates, drafts, and timeline entries for emergency/rescue content.', 'files': 'lib/page/rescue_screen/*, lib/controller/rescue_controller.dart, rescue_detail_controller.dart, lib/api/repository/rescue_repository.dart', 'flow': 'Rescue dashboard → Create rescue → Attach files/timeline → Moderate rescue updates (pending/approved).'}

General Posts Module

{'desc': 'General content posts with categories, drafts, and detail views.', 'files': 'lib/page/general_screen/*, lib/controller/general_controller.dart, general_detail_controller.dart, lib/api/repository/general_repository.dart', 'flow': 'General dashboard → Create/filter posts → Draft management → Detail view at /general/generalDetails.'}

Reports Module

{'desc': 'Review and act on reported posts, comments, and users.', 'files': 'lib/page/reports_screen/*, lib/controller/report_controller.dart, lib/api/repository/report_repository.dart', 'flow': 'Sub-tabs for Posts / Comments / Users → View report details → Delete comment or block user.'}

User Management Module

{'desc': 'Search and manage app users; view profiles; block/unblock users.', 'files': 'lib/page/user_management_screen/*, lib/page/user_profile_screen/, lib/controller/user_management_controller.dart, lib/api/repository/user_manage_repository.dart', 'flow': 'Paginated user list (16 per page) → Search → Open profile dialog → Block/unblock via API.'}

Super Admin Module

{'desc': 'Owner-role admins approve or reject pending admin registrations.', 'files': 'lib/page/super_admin_dashboard/, lib/controller/super_admin_controller.dart, lib/api/repository/super_admin_repository.dart', 'flow': 'Login with role=owner → Super Admin tab → List admins by filter → Approve/Reject. Route: /superAdminDashboard (also accessible as dashboard tab 4).'}

Onboarding Module

{'desc': 'First-time admin setup including location radius configuration.', 'files': 'lib/page/onboarding_screen/, lib/controller/onboarding_controller.dart', 'flow': 'Post-registration or first login → Onboarding screens → Location/radius setup.'}

10. UI/UX Structure

Navigation Approach

GoRouter (^7.0.0) integrated via GetMaterialApp.router.

Routes defined in lib/route/app_route.dart.

Navigation helpers in lib/route/navigator_route.dart (context.push, context.go, context.pop).

Legacy GetX routing (GetPage) is commented out.

Auth middleware (AuthMiddleware) exists for GetX but is NOT currently applied to GoRouter routes — auth is handled manually in SplashScreen.

Theme Management

- Dark theme enforced in main.dart ThemeData
- AppColors in lib/theme/colors.dart (black scaffold, drawer #121212, accent blues)
- Font family: Poppins (primary), Arvo (secondary)
- Material 3 enabled (useMaterial3: true)
- ToastificationWrapper for global toast overlay

Responsive Design

- flutter_screenutil with designSize: Size(1920, 1080)
- Responsive utility (lib/utils/responsive_utils.dart):

- Mobile: width < 850px

- Tablet: 850px – 1099px

- Desktop: width ≥ 1100px

- Desktop: permanent side drawer; Mobile: collapsible drawer via Scaffold.drawer
- textScaler fixed to linear(1) to prevent system font scaling issues

11. Assets & Resources

Images

assets/image/ — logos (aawaz_logo.svg, logo.png), map background, profile placeholders, category illustrations (fire, police, hospital, traffic, etc.)

Icons

assets/icons/ — SVG and PNG icons (dashboard, search, filter, rescue, reports, user management, etc.)

Animations

assets/animation/ — loader.json, sucess_animation.json (Lottie)

Fonts

font/ — Poppins (Light, Regular, Medium, SemiBold, Bold), Arvo Bold

Registered in pubspec.yaml and referenced via ThemeData.fontFamily

Localization

flutter_intl enabled in pubspec.yaml.

lib/l10n/intl_en.arb — empty (no strings defined).

lib/generated/l10n.dart — generated but unused.

Status: Localization infrastructure present; all UI strings are hardcoded in English.

Asset Code Generation

flutter_gen generates lib/gen/assets.gen.dart — use Assets.image.*, Assets.icons.* for type-safe asset access.

12. Testing & QA

Unit Tests

Not Available — no unit test files in test/ directory.

Widget Tests

test/widget_test.dart exists but contains default Flutter counter smoke test (expects '0' and '+' icon) — incompatible with current MyApp. Test will fail if run.

Integration Tests

Not Available

Known Test Coverage

Effectively 0% for Admin Web. No CI test pipeline configured.

Manual QA Recommendations

- Login/logout flows (email + Google)
- Event create → draft → publish → filter
- Rescue update moderation
- Report actions (delete comment, block user)
- Super Admin approve/reject
- Responsive layout at 1920, 1024, and 375px widths
- Firebase Hosting SPA routing (deep links refresh correctly)

13. Performance Considerations

Optimization Techniques

- cached_network_image for network image caching
- Pagination on event lists and user lists (pageNumber increment)
- Debounced location search (Timer debounce in DashboardController)
- HTML web renderer for broader browser compatibility (vs CanvasKit)
- flutter_screenutil for consistent scaling without manual MediaQuery math

Memory Management

- ScrollControllers disposed in controller dispose() methods
- Debounce timers cancelled on dispose
- GetStorage for lightweight persistence vs heavy local DB
- Large file uploads use MultipartFile.fromBytes with size considerations

Best Practices Followed

- Repository pattern separates API from UI
- Centralized API endpoints and headers
- Type-safe asset generation via flutter_gen
- Separation of page components into subfolders

Areas for Improvement

- EventController is very large (~1200+ lines) — consider splitting
- Multiple Get.put() calls for same controller across widgets — risk of duplicate instances
- Token interceptor commented out in ApiControllerMain

14. Known Issues / Limitations

Bugs / Incomplete Features

- GoRouter auth redirect is commented out — unauthenticated users can manually navigate to /dashboard
- AuthMiddleware designed for GetX routing, not integrated with GoRouter
- SplashController navigation logic is commented out (handled in SplashScreen widget instead)
- widget_test.dart is outdated and will fail
- NavigatorRoute.navigateToReplacement still uses Get.offAndToNamed (inconsistent with GoRouter)

- Super Admin dashboard route exists but owner users are redirected to /dashboard (tab 4) instead of dedicated route
- Hardcoded Radar.io API key in dashboard_controller.dart (security concern)
- Commented hardcoded JWT in api_endpoint.dart requestHeader (must never be enabled in production)

Pending Features

- Full localization (intl_en.arb is empty)
- Automated test suite
- Environment-based config (.env / flavors for dev/staging/prod)
- GoRouter auth guard implementation

Technical Debt

- Legacy consumer app code (lib/presentation/, lib/core/, lib/data/) clutters repository
- Mixed navigation APIs (GoRouter + residual GetX navigation)
- Typo naming conventions (dailoge, feild, calander) throughout codebase
- Duplicate model definitions (lib/model/ vs lib/data/models/)
- secrets.json and secrets.properties in working tree — verify .gitignore coverage

15. Future Improvements

Suggested Enhancements

- Implement GoRouter redirect for authentication and role-based access
- Extract environment variables (API URL, Radar key, Firebase config) to secure config
- Add comprehensive widget and integration tests
- Remove or isolate legacy consumer app code into separate package
- Implement proper error boundary and global error handler
- Add audit logging for admin actions (approve, block, delete)
- Enable CanvasKit or Wasm renderer evaluation for web performance
- Complete i18n with intl code generation

Scalability Improvements

- Split large controllers into use-case services
- Introduce repository interfaces for testability
- Add API response caching layer for categories/reactions
- Implement proper state management per module with GetX bindings
- Consider feature modules with lazy loading for web bundle size
- Set up CI/CD pipeline (build web → Firebase deploy on merge)

16. Credentials & Access

Firebase Project

Project ID: eagle-eye-oprator | Hosting: Firebase Hosting (build/web)

API Base URLs

Production Admin API: https://awaazeye.com/admin/v1 | User API: https://awaazeye.com/api/v1

Google Sign-In (Web)

Client ID: 1086312563652-****.apps.googleusercontent.com (see web/index.html)

Firebase Web API Key

AIzaSy**** (see web/index.html and lib/firebase_options.dart) — restrict by domain in Firebase Console

Radar.io API Key

prj_live_pk_**** (hardcoded in dashboard_controller.dart) — rotate and move to secrets

Admin Access Notes

- Admin accounts require backend registration and approval (Super Admin for owner role)
- Role 'owner' enables Super Admin tab and isSuperAdmin flag in local storage
- JWT Bearer token required for all protected API endpoints
- Do NOT commit production tokens or API keys to version control

Secrets Files

config/secrets.json, android/secrets.properties, ios/Flutter/Secrets.xcconfig — verify access with DevOps; mask in documentation

17. Handover Notes

Important Developer Notes

- This is a Flutter Web-first admin panel; always test changes in Chrome at 1920×1080
- Use flutter build web --web-renderer html --release before Firebase deploy
- logic_go_network is a private Bitbucket dependency — ensure team access
- Primary active code paths: main.dart → AppRoutes.router → lib/page/*
- Ignore lib/presentation/ for Admin Web unless explicitly migrating features
- Run dart run build_runner build after adding/modifying assets
- Package name in pubspec is eagle_eye_admin; app title is 'Awaaz Operator'

Things to Be Careful About

- Do not enable commented hardcoded JWT in api_endpoint.dart
- Auth is NOT enforced at router level — always implement redirect before production hardening
- Get.put() in multiple widgets can cause state inconsistency — prefer Bindings or single put at route level
- File uploads differ between web (bytes) and mobile (file path) — test both if supporting mobile
- Firebase messaging on web requires service worker (firebase-messaging-sw.js)
- Clearing storage on logout must preserve 'keep me logged in' credentials per AppFunctions.onTokenExpire pattern
- Before handover to client: rotate any exposed API keys found in source code

Document Metadata

Generated: June 17, 2026 | Document Version: 1.0 | Scope: Awaaz Admin Web Panel